

Django Installation -- Complete Notes

1. How do I get started with Django?

The general steps to get started with Django are:

1. Install Python.
2. Create a virtual environment.
3. Activate the virtual environment.
4. Install Django.
5. Create a Django project.
6. Run the development server.
7. Follow the Django tutorial.
8. Continue with the Django documentation and advanced topics.

2. Django Prerequisites

Django is a Python web framework, so Python is required.

For development, you generally need:

- * Python
- * Virtual Environment
- * Django
- * SQLite

You do **not** need to install Apache, Nginx, or a separate database server just to learn or experiment with Django.

Django includes a lightweight development server.

To run the development server:

```
```bash
```

```
python manage.py runserver
```

```
```
```

```
---
```

3. Which Python Versions Does Django Support?

| Django Version | Supported Python Versions |
|----------------|---------------------------|
|----------------|---------------------------|

| | |
|-------|-------|
| ----- | ----- |
|-------|-------|

| | |
|------------|-----------------------------------|
| Django 4.2 | Python 3.8, 3.9, 3.10, 3.11, 3.12 |
|------------|-----------------------------------|

| | |
|------------|-------------------------|
| Django 5.0 | Python 3.10, 3.11, 3.12 |
|------------|-------------------------|

| | |
|------------|-------------------------------|
| Django 5.1 | Python 3.10, 3.11, 3.12, 3.13 |
|------------|-------------------------------|

| | |
|------------|-------------------------------------|
| Django 5.2 | Python 3.10, 3.11, 3.12, 3.13, 3.14 |
|------------|-------------------------------------|

| | |
|------------|-------------------------|
| Django 6.0 | Python 3.12, 3.13, 3.14 |
|------------|-------------------------|

For each Python series, the latest micro release is officially supported.

Which Python version should I use?

Generally, use the latest supported Python 3 version.

However, when choosing a Python version for a project, consider:

```
```text
```

Django Version
----------------

Supported Python Version
--------------------------

Third-party Package Compatibility
-----------------------------------

```
```
```

```
---
```

4. Stable Version or Development Version?

For normal projects and production applications, use a **stable Django release**.

Stable Version

```text

Stable Django

Production

Recommended

```

Development Version

```text

Development Django

Testing / Contribution

Not normally recommended for production

```

The development version is mainly useful for testing upcoming features or contributing to Django itself.

5. Install Python

First, install Python.

Official Python website:

<https://www.python.org/downloads/>

After installing Python, check the version:

```
```bash
python --version
```
```

On Windows, you can also use:

```
```bash
py --version
```
```

Example:

```
```text
Python 3.13.7
```
```

6. What is a Virtual Environment?

A virtual environment is an **isolated Python environment**.

It keeps the packages and dependencies of one project separate from another project.

For example:

```
```text
Project A
venv
Django 5.2
```
```

Project B

venv

Django 6.0

This allows each project to use its own package versions.

Therefore, using a virtual environment is highly recommended for Django projects.

7. Create a Virtual Environment

First, create a project directory:

```
```bash
mkdir myproject
cd myproject
```
```

Create a virtual environment:

```
```bash
python -m venv venv
```
```

Here:

```
```text
python
```

Python interpreter

```
-m venv
```

Use the venv module

```
venv
```

Name of the virtual environment

```
```
```

```
---
```

8. Activate the Virtual Environment

Windows CMD

```
```bash
```

```
venv\Scripts\activate
```

```
```
```

Windows PowerShell

```
```powershell
```

```
.\venv\Scripts\Activate.ps1
```

```
```
```

Linux / macOS

```
```bash
```

```
source venv/bin/activate
```

```
```
```

After activation, you will normally see something like:

```
```text
```

```
(venv) C:\Users\Username\myproject>
```

---

`(venv)` means that the virtual environment is active.

---

## # 9. Install Django

After activating the virtual environment, install Django:

```
```bash
python -m pip install Django
```
```

This is the recommended way to install Django with pip.

Check whether Django was installed successfully:

```
```bash
python -m django --version
```
```

You can also use:

```
```bash
django-admin --version
```
```

Example:

```
```text
6.0
```
```

---

## # 10. What is pip?

`pip` is Python's package installer.

It is used to install, upgrade, and manage Python packages.

Example:

```
```bash
python -m pip install Django
```
```

Here:

```
```text
python
```

Python interpreter

-m pip

Use the pip module

install

Install command

Django

Package to install

```
```
```

```

```

## # 11. Django Database Support

Django officially supports several databases:

- \* PostgreSQL
- \* MariaDB
- \* MySQL
- \* SQLite
- \* Oracle

---

## # 12. SQLite

SQLite is a simple database option for Django projects.

It does not require a separate database server.

The database is normally stored as a file:

```
```text
db.sqlite3
```
```

SQLite is very convenient for:

- \* Learning
- \* Testing
- \* Small projects
- \* Prototyping

---

## # 13. Which Database Should I Use in Production?

For production applications, Django documentation recommends PostgreSQL.

A common approach is:

```
```text
```

Learning

SQLite

Small Project

SQLite / PostgreSQL

Production

PostgreSQL

```
```
```

However, the final database choice depends on the requirements of the application.

```

```

## # 14. Installing Database Drivers

If you use a database other than SQLite, you may need a Python database driver.

### ## PostgreSQL

You can install:

```
```bash
```

```
pip install psycopg2
```

```
```
```

### ## MySQL / MariaDB

You can install:

```
```bash
```

```
pip install mysqlclient
```

```
```
```

```
Oracle
```

You can install:

```
```bash
```

```
pip install oracledb
```

```
```
```

```

```

## # 15. Django Database Configuration

Django's database configuration is located in:

```
```text
```

```
settings.py
```

```
```
```

The default SQLite configuration looks like:

```
```python
```

```
DATABASES= {
```

```
"default": {
```

```
"ENGINE": "django.db.backends.sqlite3",
```

```
"NAME": BASE_DIR / "db.sqlite3",
```

```
}
```

```
}
```

```
```
```

---

## # 16. ENGINE

The `ENGINE` setting specifies which database backend Django should use.

### ### SQLite

```
```python
```

```
"ENGINE": "django.db.backends.sqlite3"
```

```
```
```

### ### PostgreSQL

```
```python
```

```
"ENGINE": "django.db.backends.postgresql"
```

```
```
```

### ### MySQL

```
```python
```

```
"ENGINE": "django.db.backends.mysql"
```

```
```
```

### ### Oracle

```
```python
```

```
"ENGINE": "django.db.backends.oracle"
```

```
```
```

---

## # 17. NAME

For SQLite:

```
```python
"NAME": BASE_DIR / "db.sqlite3"
```
```

This means the SQLite database file will be stored inside the project directory as:

```
```text
db.sqlite3
```
```

You do not need to manually create the SQLite database file.

Django will create it when needed.

---

## # 18. Configuration for Other Databases

For databases other than SQLite, you normally need additional settings.

Example for PostgreSQL:

```
```python
DATABASES= {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mydatabase",
        "USER": "myuser",
        "PASSWORD": "mypassword",
        "HOST": "localhost",
    }
}
```

```
"PORT": "5432",  
}  
}  
...
```

The main settings are:

```
```text
```

ENGINE

Database backend

NAME

Database name

USER

Database user

PASSWORD

Database password

HOST

Database server

PORT

Database server port

```
```
```

```
---
```

19. Creating a Database

When using a database such as PostgreSQL or MySQL, the database itself may need to be created first.

For example:

```
```sql
CREATE DATABASE database_name;
```
```

Then configure the database connection in Django's `settings.py`.

20. What is Migration?

Django's migration system is used to create and modify database tables based on your models.

The main migration command is:

```
```bash
python manage.py migrate
```
```

The basic workflow is:

```
```text
```

Django Models

Migrations

Database Tables

```
```
```

21. Is Apache Required for Django?

No.

Apache is ****not required when learning or developing with Django****.

Django provides its own lightweight development server.

Therefore, beginners can start with:

```
```text
```

Python

+

Virtual Environment

+

Django

+

SQLite

```
```
```

22. Production Web Server

In production, Django can work with different server configurations.

A common architecture is:

```
```text
```

Client

Nginx / Apache

Gunicorn / uWSGI

Django

Database

...

Django supports both **WSGI** and **ASGI**.

WSGI stands for:

```text

Web Server Gateway Interface

...

23. Apache + mod_wsgi

Django can be deployed with Apache using `mod_wsgi`.

`mod_wsgi` provides two modes:

```text

1. Embedded Mode

2. Daemon Mode

...

### Embedded Mode

Python code runs inside the Apache process.

### Daemon Mode

A separate daemon process handles Django requests.

The appropriate deployment setup depends on the project's requirements and infrastructure.

---

## # 24. Complete Django Installation Process

The complete workflow is:

``text

Step 1

Install Python

Step 2

Create Project Directory

Step 3

Create Virtual Environment

Step 4

Activate Virtual Environment

Step 5

Install Django

Step 6

Create Django Project

Step 7

Run Migration

Step 8

Run Development Server

...

---

## # 25. Practical Django Installation -- Windows

Suppose we want to create a Django project named `myproject`.

### ## Step 1: Create Project Directory

```
```bash
mkdir myproject
cd myproject
```
```

### ## Step 2: Create Virtual Environment

```
```bash
python -m venv venv
```
```

### ## Step 3: Activate Virtual Environment

```
```bash
venv\Scripts\activate
```
```

### ## Step 4: Install Django

```
```bash
python -m pip install Django
```
```

### ## Step 5: Check Django Version

```
```bash
python -m django --version
```
```

## ## Step 6: Create Django Project

```
```bash  
django-admin startproject config .  
```
```

The `` at the end is important.

Project structure:

```
```text  
myproject/  
  
manage.py  
  
config/  
__init__.py  
settings.py  
urls.py  
asgi.py  
wsgi.py  
  
venv/  
```
```

---

## # 26. Run Migrations

Run:

```
```bash  
python manage.py migrate
```

This applies Django's default migrations to the database.

27. Run the Development Server

Run:

```
```bash
python manage.py runserver
```
```

Then open the following address in your browser:

```
```text
http://127.0.0.1:8000/
```
```

You should see the Django welcome page.

28. Complete Commands

The most important commands for starting a Django project are:

```
```bash
mkdir myproject
cd myproject

python -m venv venv

venv\Scripts\activate
```

```
python -m pip install Django
```

```
python -m django --version
```

```
django-admin startproject config .
```

```
python manage.py migrate
```

```
python manage.py runserver
```

```
...
```

```

```

## # 29. Development vs Production

Feature	Development	Production
Web Server	Django `runserver`	Nginx/Apache + WSGI/ASGI Server
Database	SQLite	PostgreSQL commonly used
Debug	Development use   `DEBUG=False`	
Security	Basic configuration	Strict security configuration
Performance	Testing	Optimized
Server	`runserver`	Production server stack

**\*\*Important:\*\*** Django's `runserver` should not be used as a production server.

```

```

## # 30. Installing the Development Version of Django

If you want to work with Django's development source code, first clone the repository:

```
```bash
```

```
git clone https://github.com/django/django.git
```

```
'''
```

This creates a directory named `django`.

Then, inside your activated virtual environment, run:

```
```bash
```

```
python -m pip install -e django/
```

```
'''
```

Here:

```
```text
```

```
-e
```

Editable Installation

```
'''
```

This makes Django's source code directly importable.

To update the Django source code:

```
```bash
```

```
cd django
```

```
git pull
```

```
'''
```

```

```

### # 31. What Should a Beginner Follow?

If you are learning Django, you do not need to use the development version.

A good learning path is:

```
```text
```

Python

Virtual Environment

Stable Django

SQLite

Django Tutorial

Models

Views

Templates

Forms

Authentication

Django REST Framework

API Development

Deployment

```
```
```

```

```

# 32. Most Important Commands

### Create Virtual Environment

```
```bash
```

```
python -m venv venv
```

...

Activate on Windows

```bash

venv\Scripts\activate

...

### Activate on Linux/macOS

```bash

source venv/bin/activate

...

Install Django

```bash

python -m pip install Django

...

### Check Django Version

```bash

python -m django --version

...

Create Django Project

```bash

django-admin startproject config .

...

### Run Migration

```
```bash
```

```
python manage.py migrate
```

```
```
```

```
Run Development Server
```

```
```bash
```

```
python manage.py runserver
```

```
```
```

```

```

### # 33. Short Summary

Django is a Python-based web framework.

The basic requirements for learning Django are:

```
```text
```

Python

Virtual Environment

Django

```
```
```

For development, SQLite is an easy database option because it does not require a separate database server.

For production, PostgreSQL is commonly recommended.

When learning Django, you do not need to start with Apache, Nginx, uWSGI, or other deployment technologies. Learn those later when you study deployment.

The most important installation commands are:

---

```
```bash
python -m venv venv
venv\Scripts\activate
python -m pip install Django
django-admin startproject config .
python manage.py migrate
python manage.py runserver
```
```